КАКИЕ МОГУТ БЫТЬ CI/CD

1 кейс

стартап, до 10-20 человек
приложение = один сервис на java/go/python

ТИПИЧНЫЙ СТЕК

Git: GitHub / GitLab / Gitea
CI (если вообще есть): GitHub Actions или GitLab CI
инфра: 1-3 VPS в Selectel / Yandex Cloud / Cloud.ru и т.п.

запуск приложения:
- либо через docker-compose
- либо вообще по старинке через systemd + java -jar / python app.py / бинарь go

бд: PostgreSQL (или managed, или докер контейнер на том же VPS)
мониторинг: либо ничего, либо Grafana+Prometheus на отдельном/том же VPS
логи: docker logs или файловые логи + tail -f

КАК ПРОИСХОДИТ РАЗРАБОТКА

есть main или master ветка - то, что сейчас в проде (или почти)
когда пишут фичи:
- либо сразу в master пушат
- либо делаю все-таки ветку от мастера - feature/epta-fi4a

допустим разраб все таки сделал ветку, написал код, локально запустил тесты (если они есть)
запушил в репу, дай Бог он создаст Merge Request / Pull Request в master (запрос на слияние веток)
и потом заапрувит и замержит

ЧТО ТАМ С CI/CD

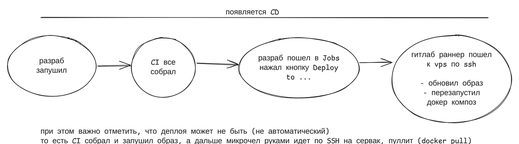
несколько вариантов

1) ci - вообще нет

2) минимальный CI

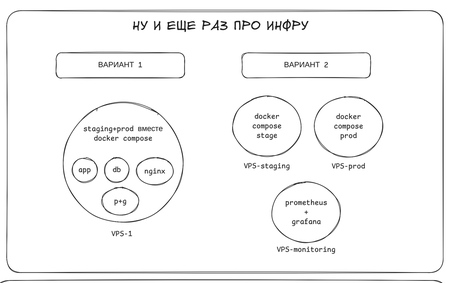
при этом важно отметить, что деплоя может не быть (не автоматический)
то есть CI собрал и запушил образ, а дальше микрочел руками идет по SSH на сервак, пуллит (docker pull)
новый образ и перезапускает docker-compose

3) есть и CI и CD, ну с ручным запуском



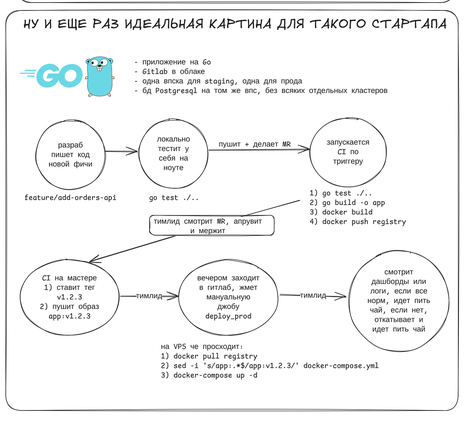
при этом важно отметить, что деплоя может не быть (не автоматический)
то есть CI собрал и запушил образ, а дальше микрочел руками идет по SSH на сервак, пуллит (docker pull)
новый образ и перезапускает docker-compose

НУ И ЕЩЕ РАЗ ПРО ИНФРУ



НУ И ЕЩЕ РАЗ ИДЕАЛЬНАЯ КАРТИНА ДЛЯ ТАКОГО СТАРТАПА

- приложение на Go
- Gitlab в облаке
- одна впска для staging, одна для прода
- бд Postgresql на том же впс, без всяких отдельных кластеров



2 кейс

средняя РФ компания, 50-200 человек
несколько сервисов или жирный монолит + пара вспомогательных

ТИПИЧНЫЙ СТЕК

Git: чаще всего GitLab (онпрем или клауд)
CI/CD - Gitlab CI или Jenkins, иногда в паре - Jenkins для деплоя, Gitlab для кода
инфра: кубер-кластер в клауде, либо онпрем vSphere/Proxmox, 10-30 нод

деплой приложений:
- хелм чарты
- голые манифесты кубера
- иногда уже заезжает ArgoCD

бд: Postgres, Redis
иногда есть кликхаус под отчеты/аналитику

мониторинг: prometheus + grafana, alertmanager мб

логи:
- либо ELK
- либо Loki
- плюс kubectl logs никто не отменял

КАК ПРОИСХОДИТ РАЗРАБОТКА

- есть master ветка - то что было/будет в проде
- есть develop ветка - сливается все новое
- под фичи делают ветку например с номер таски в жире
- feature/DEV-431-add-payment-handler
- bugfix/DEV-421-fix-user-session
- hotfix/DEV-987-prod-fix

хорошая практика (не всегда она есть) - нельзя просто так пушить в master
- на уровне гитлаба стоят защищенные ветки
- пуш только через MR (мерж реквест)
- иногда у руководителей есть права запушить прям в мастер

ЖИЗНЕННЫЙ ЦИКЛ ФИЧИ

1) разраб берет задачу в jira
2) создает ветку от master или develop
3) пишет код, локально гоняет тесты
4) пушит ветку в гитлаб
5) создает Merge Request
- указывает номер таски
- кратко описывает, что сделал
6) CI запускается по вебхуку
7) если CI зеленый - кто-то делает код ревью и апрувит
8) после апрува MR мержится в master или develop
9) на master или develop запускается второй пайплайн

ТУТ МОЖЕТ ВОЗНИКНУТЬ ВОПРОС, ЗАЧЕМ НАМ ДВА ПАЙПА?
- первый пайп который триггерится на мердж реквест - нужен, чтобы не влить в целевую ветку совсем хуевый код

в нем че делается
- build - сборка
- юнит тесты
- линтеры
- иногда простой интеграционный тест

то есть тут нет деплоя, основная цель - собрать образ для проверки

а во втором пайпе уже происходит деплой
какие стейджи:
1) снова сборка + тесты
2) сборка докер образа с нормальным тегом app:1.2.3 или app:master-sha (ветка-хэш коммита)
3) пуш в registry (Gitlab Registry, Nexus, Artifactory)
4) деплой

типовой набор стендов для таких компаний
- dev - песочница, потыкать, поковырять
- staging - приближенная к проду среда
- prod - ну прод есть прод

БАЗА ПО ДЕПЛОЮ

наш стартер пак
- Gitlab
- Kubernetes
- несколько стендов : dev -> stage -> prod
- один и тот же образ, просто на разных окружения (вальюсы, енвы, секреты)

Что считаем стендом?

- один кластер k8s, в нем неймспейсы:
- app-dev
- app-staging
- app-prod
- либо несколько кластеров
- k8s-dev
- k8s-staging
- k8s-prod

но мы пока не будем усложнять, остановимся на один кластер

Что происходит до деплоя (билд + образ)

сначала общий шаг, ОДИН РАЗ:
1) разраб замержил MR в develop
2) запустился пайплайн на ветке develop
3) пайплайн делает
- билд (maven, go build, pytest)
- docker build -> образ в registry.company.ru/myapp:<commit-sha>
- docker push
ДАЛЬШЕ НА ВСЕХ СТЕНДАХ, используют один и тот же образ, просто с разными настройками

Как обычно деплой в дев устроен

- после того, как образ собран и запушен, в пайпе есть джоба, пример:

```
deploy_dev:
stage: deploy
needs: ["build"] # ждём сборку
when: on_success
script:
- helm upgrade --install myapp ./helm \
-n myapp-dev \
-f ./helm/values-dev.yaml \
--set image.tag=$CI_COMMIT_SHA
```

как это происходит:
1) гитлаб раннер или дженкинс агент получает джобу - deploy_dev
2) у раннера есть доступ к кластеру:
- либо через kubeconfig (секрет, который лежит в CI)
- либо раннер сам крутится в k8s и юзает Service Account
3) джоба вызывает helm upgrade --install
4) helm:
- смотрит чарт ./helm
- берет вальюсы из файлика values-dev.yaml
- подставляет image.tag=$CI_COMMIT_SHA
- обновляет деплоймент, сервис, ингресс в неймспейсе app-dev
5) кубер делает роллинг алдейт
- поднимает новые поды с новым образом
- ждет пробы
- гасит старые поды

что тут важно понимать:
- dev почти всегда деплоится автоматически после мерджа в master (или другую ветку)
- в деве могут жить сырые фичи, которые еще особо не тестили
- QA и разрабы могут смотреть изменения и вот это все

Staging - деплой либо по тегу, либо по кнопке

вариант 1 - деплой на stage по кнопке (manual job)

```
deploy_staging:
stage: deploy
needs: ["build"]
when: manual # ручной запуск
script:
- helm upgrade --install myapp ./helm \
-n myapp-staging \
-f ./helm/values-staging.yaml \
--set image.tag=$CI_COMMIT_SHA
```

1) разработчик/тимлид/devops смотрит на dev - все ок
2) открывает пайплайн -> жмет кнопку deploy_staging
3) job делает то же самое, что и на dev, но:
- namespace: app-staging
- values: values-staging.yaml (другие БД, другие лимиты, другие урлы сервисов)
- helm обновляет приложение в app-staging

вариант 2 - деплой на stage по тегу (релизный подход)
1) тимлид доволен состоянием версии на деве
2) идет в ветку, ставит тег
```
git tag v1.4.0
git push --tags
```
3) в CI есть правило:

```
deploy_staging:
stage: deploy
only:
- tags
script:
- helm upgrade --install myapp ./helm \
-n myapp-staging \
-f ./helm/values-staging.yaml \
--set image.tag=$CI_COMMIT_SHA
```

4) как только пушится тег, запускается пайп, который:
- берет образ того коммита, на котором висит тег
- деплоит его на стейджинг

Prod

в среднем по РФ прод деплоят:
- либо по тегу
- либо по кнопке
- иногда отдельным пайпом

вариант 1 - мануальная джоба из того же пайпа

```
deploy_prod:
stage: deploy
when: manual
needs: ["deploy_staging"] # опционально
script:
- helm upgrade --install myapp ./helm \
-n myapp-prod \
-f ./helm/values-prod.yaml \
--set image.tag=$CI_COMMIT_SHA
```

сценарий:
1) на стейджинг все протестировали
2) в пайпе жмут deploy_prod
3) CI залезает в кластер, в неймспейс app-prod
4) хелм обновляет деплоймент уже с
- прод конфигом
- прод секретом
- прод ресурсами

вариант 2 - отдельный релиз пайплайн

Что именно отличается между стендами

еще раз важно проговорить

код и образ - один и тот же
отличаются:
1) values файл:
- dev - мало ресурсов, тестовые бд
- staging - почти как прод, но другая БД и урлы внешний интеграций
- prod - реальные ресурсы, реальные эндпоинты и тд
2) секреты
3) ресурсы
4) фичафлаги - какие-то функции включены только на деве/стейджинге

3 кейс

банк ВТБ. масштаб всем понятен
один из примеров (как у меня было)

СТЕК
приложения:
backend: java микросервисы (15 сервисов)
frontend: node.js

git & CI/CD:
git: GitLab (self-hosted)
CI/CD: Jenkins
без вебхуков, все запускается вручную

инфраструктура:

- kubernetes кластеры в облаке
- на каждом стенде (после IFT, включительно):
~5 master нод
~20-40 worker нод
- стендин namespace (второй namespaceв том же кубер кластере для стендина)

базы данных:
- 3 команды разработки
- для каждой команды: 2 инстанса PostgreSQL
- реплики: main + standin

- kafka: 2 штуки на стенд (main + standin)
apache flink: развертывается Ansible ролью на Astra Linux (хуйня для сервисов нужная, туда тоже деплоятся некоторые сервисы)

артефакты:
- nexus (хранение Docker образов)

GIT WORKFLOW
структура репозиториев:
- каждый микросервис = отдельный репозиторий
Ветки:
master - продакшн версия
develop - самая свежая версия, сюда сливаются все фичи
feature/* - ветки для разработки фич
release/* - релизные ветки (формируются 1-2 раза в месяц)

процесс:
- разработка в feature-ветках
- слияние в develop
- формирование release-ветки 1-2 раза в месяц
- деплой релиза

СЕТЕВАЯ АРХИТЕКТУРА
контур 1: Разработка
DEV - основная разработка
ST (Staging) - стендин дева для тестов

контур 2: Интеграция
IFT - интеграционные тесты с другими сервисами
приближен к проду по ресурсам и архитектуре

контур 3: нагрузочное тестирование + препрод/прод

LT - нагрузочные тесты
UAT - препрод
PROD - продакшн

- контуры изолированы друг от друга, для переноса артефактов используются специальные агенты с доступом к обоим контурам

ЖИЗНЕННЫЙ ЦИКЛ ФИЧИ/БАГФИКСА
этап 1: локальная разработка

1) разработчик создает feature-ветку
2) разрабатывает локально
3) тестирует через Docker Compose
4) пушит изменения в свою ветку
5) создает тег в формате 2025-11-20

этап 2: деплой на DEV/ST (Контур 1)

1) разработчик идет в Jenkins
2) находит джобу: Deploy service <название>
3) нажимает "Запустить с параметрами"

Параметры джобы:
- выбор стенда: DEV или ST
действие:
- build - только сборка
- build + push to nexus - сборка + пуш в Nexus
- skip - не билдить, деплоить готовый артефакт
+ чекбокс DEPLOY_TO_KUBERNETES (если не нужно деплоить, а просто собрать, или собрать + запушить в регистри)
+ GIT_BRANCH - здесь динамически подтягиваются все ВЕТКИ И ТЕГИ (то есть можно деплоить из ветки, или из тега, как вот разраб создавал 2025-11-20)

что происходит в джобе:

1) сборка приложения (если выбрано)
2) создание Docker образа
3) пуш в Nexus (если выбрано)
4) деплой в Kubernetes через helm upgrade

результат:

1) разработчик проверяет работу в кубере
2) фиксируется артефакт (Docker образ) с тегом
3) код в репозитории зафиксирован с этим же тегом

этап 3: Перекладка на IFT (Контур 1 → Контур 2)
есть проблема: сетевые контуры изолированы друг от друга
и есть решение: специальная джоба ПЕРЕКЛАДЧИК

как работает:

1) запускается джоба-перекладчик на агентах с доступом к обоим контурам
2) идет в контур DEV/ST
забирает:
- код с нужным тегом
- docker образ из Nexus
3) перекладывает в IFT контур

Деплой на IFT:
- та же самая джоба деплоя
- те же параметры
- важность выше: всякие ебаные сбои и ошибки влияют на смежные команды, и они начинают писать в чат, вот у нас все упало, вам пиздец

этап 4: нагрузочное тестирование (Контур 2 → Контур 3)

процесс:

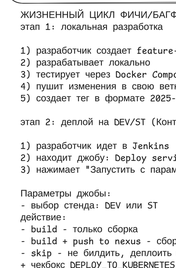
1) повторная перекладка артефактов на LT
2) деплой через ту же джобу Jenkins
3) НТшники проводят нагрузочные тесты
4) проверяем производительности под нагрузкой

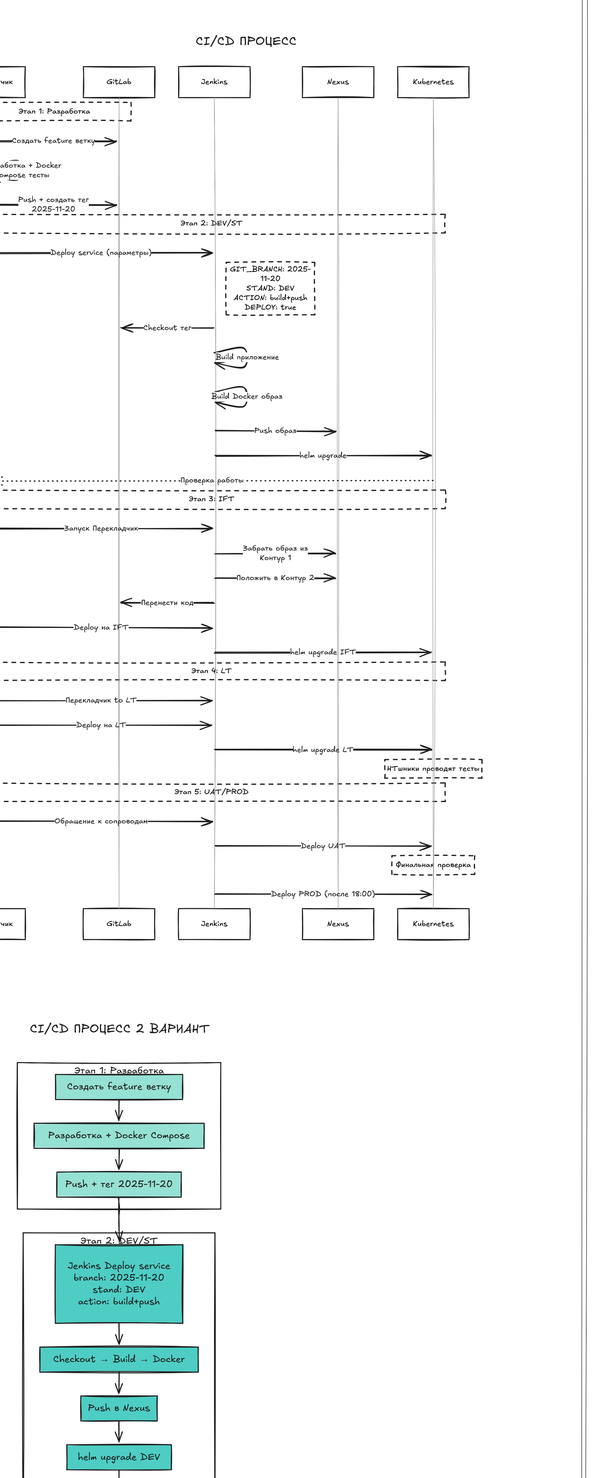
этап 5: препрод и прод (Контур 3 → Контур 4)
- если все тесты на LT прошли успешно

процесс:
1) обращение к сопроводам прода
2) деплой на UAT (препрод)
3) финальная проверка
4) деплой на PROD после 18:00 (вечернее окно)

ТЕПЕРЬ К СХЕМАМ

СТЕНДЫ



CI/CD ПРОЦЕСС

CI/CD ПРОЦЕСС 2 ВАРИАНТ